

2.1.1 Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4). Sample Test Cases
- Test case 1

1. Add ↵

2. Remove ↵

3. Display ↵

4. Quit ↵

Enter choice: 1

Integer: 11

List after adding: [11] ↵

1. Add ↵

2. Remove ↵

3. Display ↵

4. Quit ↵

Enter choice: 1

Integer: 25

List after adding: [11, 25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 26

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 11

List after removing: [25]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

Integer: 25

List after removing: []

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

List is empty

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 2

List is empty

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 8

Invalid choice

1. Add

2. Remove

3. Display

4.·Quit↵

Enter·choice:·4

- Test case 2

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·1

Integer:·1

List·after·adding:·[1]↵

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·1

Integer:·2

List·after·adding:·[1,·2]↵

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·1

Integer:·3

List·after·adding:·[1,·2,·3]↵

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·1

Integer:·4

List·after·adding:·[1,·2,·3,·4]↵

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·2

Integer:·2

List·after·removing:·[1,·3,·4]↵

1.·Add↵

2.·Remove↵

3.·Display↵

4.·Quit↵

Enter·choice:·2

Integer: 6

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

[1, 3, 4]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 4

Average time

0.020 s

19.67 ms

Maximum time

0.028 s

28.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Invalid choice

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 4

Invalid choice

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 4

Test case 2 15 ms

```
list1=["Add","Remove","Display","Quit"]
```

```
list=[]
```

```
while True:
```

```

for i in range(len(list1)):
    print(f"{i+1}. {list1[i]}")
choice = int(input("Enter choice: "))
if choice==1:
    try:
        a=int(input("Integer: "))
        list.append(a)
        print("List after adding:",list)
    except int:
        print("Invalid input")
elif choice ==2:
    if list==[]:
        print("List is empty")
        continue
    a=int(input("Integer: "))
    if a in list:
        list.remove(a)
        print("List after removing:",list)
    else:
        print("Element not found")
elif choice==3:
    if list ==[]:
        print("List is empty")
    else:
        print(list)
elif choice==4:
    break
else:
    print("Invalid choice")

```

2.1.2. Dictionary Operations

09:24

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

- Sample Test Cases

- Test case 1

```
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
```

11

Suresh

```
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
```

3

Karthik

```
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
```

5

After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}

Traversing Dictionary:

1: Amit
2: Riya
3: Karthik
4: Neha
6: Pooja
7: Rahul
8: Sneha
9: Vikram
10: Anjali
11: Suresh

- Test case 2

Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}

12

Meera

After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

6

Divya

After Update: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

1

After Deletion: {2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Divya', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 12: 'Meera'}

Traversing Dictionary:

2: Riya
3: Kiran
4: Neha
5: Arjun
6: Divya
7: Rahul
8: Sneha
9: Vikram
10: Anjali
12: Meera

Initial dictionary with 10 predefined records

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}  
  
# Display original dictionary  
print("Original Dictionary:", student)  
  
# 1. Insertion  
ins_key = int(input())  
ins_val = input()  
student[ins_key] = ins_val  
print(f"After Insertion: {student}")  
  
# 2. Update  
upd_key = int(input())  
upd_val = input()  
if upd_key in student:  
    student[upd_key] = upd_val  
print(f"After Update: {student}")  
  
# 3. Deletion  
del_key = int(input())  
if del_key in student:
```

```
student.pop(del_key)

print(f"After Deletion: {student}")
```

4. Traversal

```
print("Traversing Dictionary:")

for key, value in student.items():

    print(f"{key} : {value}")
```

2.2.1. Linear search Technique

02:02

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.
- Sample Test Cases
- Test case 1

1 2 3 4 3 5 6

3

2

- Test case 2

1 2 3 4 5 6 7 8 9 19

20

Not found

Sample Test Case:

Input:

1 2 3 4 3 5 6

3

The screenshot shows a test runner interface with the following details:

- Performance:** Average time 0.004 s (3.50 ms), Maximum time 0.006 s (6.00 ms).
- Test Results:** 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.
- Test case 1:** Passed in 4 ms. It compares 'Expected output' (1 2 3 4 3 5 6, 3, 2) with 'Actual output' (1 2 3 4 3 5 6, 3, 2).
- Test case 2:** Passed in 2 ms.
- Navigation:** Buttons for 'Terminal' and 'Test cases' are at the bottom.

Output:

```
element=input().split(" ")
```

```
ele=input()
```

```
if ele in element:
```

```
    print(element.index(ele))
```

```
else:
```

```
    print("Not found").
```

2.2.2 You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

Test case 1

171 169 185 156 174 191 186 190 187 172 160

191

```
height = map(int,input().split(" "))
```

```
print(max(height))
```

Average time
0.001 s
1.33 ms

Maximum time
0.002 s
2.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 2 ms Debug

Expected output

171 169 185 156 174 191 186 190 187 172 160

191

Actual output

171 169 185 156 174 191 186 190 187 172 160

191

Terminal Test cases